

Trabajo Practico N°4

Fecha: 6/11/2026

Estudiante: Matías Urbicain

D.N.I.: 45.740.463

Legajo: VINF018026

Comisión: Catedra C

Carrera: Licenciatura en Informática

Titular Experto: Ana Carolina Ferreyra

Titular Disciplinar: Pablo Virgolini

Materia: Seminario de Práctica de Informática

Índice

Control de versiones 2
Situación Problemática 3
Consignas 3
Desarrollo 5
1. Título 5
2. Introducción 5
3. Justificación 5
4. Definiciones del proyecto y del sistema 6
5. Elicitación 7
6. Conocimiento del negocio 7
7. Propuesta de solución 8
8. Inicio del análisis: casos de uso 10
9. Aplicación del proceso unificado de desarrollo (PUD) 16
10. Definición de la base de datos 24
11. Desarrollo del prototipo en Java 28
12. Pantallas 31
13. Versión final integradora (TP4) 35

Control de versiones

En esta sección se registran los cambios de esta entrega con respecto al Trabajo Práctico N°3. En esta versión solo se agrega el capítulo 13, que reúne todo lo nuevo del TP4.

Contenido nuevo (Trabajo Práctico N°4):

Sección	Tipo de cambio	Descripción
13. Versión final integradora (TP4)	Agregado	Nuevo capítulo con lo pedido para TP4: selección del patrón de diseño (MVC), persistencia y consulta en MySQL con JDBC, manejo de excepciones, interfaces y clases abstractas, uso de ArrayList e iteradores, y exportación de clientes a un archivo de texto. También se incluye el plan del video y un resumen de dónde se cumple cada punto de la consigna.

Situación Problemática

Los sistemas informáticos desempeñan un papel muy importante para la optimización de procesos en diversas áreas de cualquier organización. La tecnología brinda la posibilidad de automatizar tareas, recopilar y analizar datos a gran escala, agilizar procesos y proporcionar soluciones a desafíos complejos.

Te propongo que determines con claridad un problema que pueda resolverse con la implementación de un proyecto informático y realices una entrega de acuerdo con lo solicitado en la consigna. Puedes definir cualquier organización real y explorar oportunidades en relación con su seguridad, logística, gestión de inventarios,

optimización de procesos industriales, análisis de datos, cuidado de la salud, la educación, o cualquier otra área.

Para la realización de un proyecto de desarrollo informático, se requiere realizar un profundo análisis del negocio y comprender la dinámica de la organización. De esta manera, será posible comprender correctamente las necesidades, identificar los procesos, flujos de trabajo y desafíos que la organización enfrenta a diario y que pueden optimizarse con la aplicación de un desarrollo. Servirá también como base para justificar el proyecto, definir el alcance y objetivos.

Es clave llevar adelante un correcto proceso de elicitación, que involucra la definición de requerimientos funcionales y no funcionales.

Una vez completada esta fase, se avanza en el análisis y diseño detallado del sistema.

Esto implica la creación de modelos, la definición de la arquitectura, la planificación de la estructura de datos y la lógica de funcionamiento. Solo con un correcto análisis y diseño se puede garantizar que el sistema se construirá de manera eficiente y que cumplirá con los objetivos establecidos.

Consignas

Considera que continúas trabajando en la organización cuyo problema definiste resolver con un proyecto informático, y estás en el equipo de desarrollo que va a llevar adelante el mismo.

Recuerda que el proyecto informático definido debe ser distinto al considerado en la lectura y que el entregable final debe cumplir con los siguientes objetivos:

- Realizar el análisis del modelo de negocios para definir y justificar el proyecto.
- Aplicar el proceso unificado de desarrollo (PUD) para garantizar la calidad, escalabilidad y eficiencia en el ciclo de desarrollo.
- Utilizar una base de datos MySQL para la persistencia de los datos.
- Emplear Java como lenguaje de programación para el desarrollo del sistema.

A los fines del trabajo, para la implementación de la base de datos y el desarrollo con Java, puedes presentar un prototipo, que es “es la creación de un modelo operacional que incluya solo algunas características del sistema final” (Kendall & Kendall, 2011).

El concepto de operacional es clave, ya que no se trata de un simple modelo, sino que permite desarrollar módulos que se van integrando en la versión final del sistema.

Esta cuarta actividad te permitirá avanzar con el proyecto integrador que debe utilizar un patrón de diseño coherente con las definiciones establecidas en la arquitectura definida.

Para el desarrollo del prototipo en Java, deberás crear un paquete cuyas clases permitan realizar la persistencia en una base de datos MySQL. Esta actividad incluye establecer conexiones, realizar consultas, actualizar registros y presentar resultados en la interfaz. En todos los casos que se requiera, recuerda aplicar correctamente el manejo de excepciones.

El desarrollo debe incluir la aplicación de clases abstractas o interfaces. También debe contemplar la utilización de arreglos y emplear la clase ArrayList, ambas estructuras de forma complementaria.

Según las características del proyecto, puedes utilizar archivos. Aunque esta actividad es opcional, te recomiendo practicar su aplicación, previo a presentarte en el EFIP I.

Los entregables son los siguientes:

- Presentación del desarrollo en Java.
- Entrega del proyecto integrador, completo.
- Presentación del proyecto en un video con una duración de aproximadamente 3 minutos.

El proyecto o prototipo desarrollado en Java debe contemplar necesariamente las siguientes características:

- Corrección de las observaciones realizadas en la entrega de las actividades anteriores.
- Selección del patrón de diseño y justificación.
- Persistencia y consulta de datos en una base de datos MySQL. Esta actividad incluye establecer conexiones, realizar consultas, actualizar registros y presentar resultados en la interfaz.
- Correcta aplicación de excepciones para la interacción con la base de datos MySQL.
- Inclusión pertinente de clases abstractas o interfaces.
- Utilización complementaria de arreglos y de la clase ArrayList.

Desarrollo

1. Título

Desarrollo de un Sistema de Gestión Transaccional de Reservas para la Estación de Descanso ***'Buena Caza'*** en Pehuén Co.

2. Introducción

En la industria turística actual, los sistemas de información han dejado de ser herramientas de apoyo para convertirse en la base de la eficiencia operativa.

El complejo '***Buena Caza - Estación de Descanso***' es un emprendimiento familiar ubicado en la ciudad balnearia de Pehuén Co, sur de la Provincia de Buenos Aires. Las cabañas se encuentran estratégicamente forestadas y cuenta con 2 unidades habitacionales para 2, 3 y 4 personas llamadas Waingunga y Seeonee. La infraestructura actual ofrece servicios como climatización, WiFi, Smart TV y servicio de ropa blanca.

Acorde con el alto nivel de servicios que se brinda, un buen sistema informático de reserva y gestión acompaña el buen nivel del negocio.

3. Justificación

La justificación del proyecto sirve para mostrarle a los dueños que la propuesta es viable y que de verdad vale la pena invertir en esta mejora. No queremos armar un software nuevo solo para modernizar el negocio, sino para resolver los problemas reales que detectamos. Al final, solucionar estas fallas operativas ayuda directamente a que el complejo funcione mejor y a que los huéspedes se lleven una mejor imagen del lugar.

Un beneficio de la automatización son las tarifas dinámicas. Actualmente, la gestión manual de temporadas (Alta, Media y Baja) genera errores de facturación. Al definir estas reglas, se garantiza una exactitud del 100% en el cálculo de estadías.

Asimismo, la aplicación del descuento para la 'Comunidad Buena Caza' dejará de depender de controles manuales, asegurando que el beneficio de fidelidad se aplique correctamente basándose en el historial del invitado.

También permitirá que el usuario final pueda elegir y reservar las fechas sin necesidad de contactar a los dueños como hoy día se debe hacer.

4. Definiciones del proyecto y del sistema

4.1 Objetivo General del Proyecto

Desarrollar e implementar un sistema de gestión integral para el complejo de cabañas ***Buena Caza*** en la ciudad balnearia de Pehuén-Co que automatice las reservas, la fidelización de clientes y gestione las tarifas según sea temporada alta, media o baja. Asi mismo brindar a los dueños un reporte de la situación global.

4.2 Objetivos Específicos

- Centralizar la administración de ocupación.
- Automatizar la validación de descuentos de fidelidad asociados a las reservas previas.
- Facilitar la consulta de disponibilidad en tiempo real para el perfil de usuario "Invitado".

4.3 Objetivos del Sistema

El sistema se clasifica bajo dos categorías de los sistemas de información:

- TPS (Transaction Processing System): Definido como un sistema que registra "transacciones comerciales consumadas" ya que el software procesará las reservas asegurando la integridad de los datos.
- MIS (Management Information System): Proporcionará información dirigida a los dueños con reportes.

4.4 Alcances y límites

A continuación, un listado del alcance y los límites

- el sistema incluirá perfiles diferenciados (Administrador/invitado)
- gestión de las 2 cabañas
- un total de 8 pantallas
- incluirá 2 reportes de listado de invitados y de ocupación de las cabañas
- el sistema no gestionará pasarelas de pago externas
- no incluye ningún equipo o hardware

4.5 Restricciones Tecnológicas

El desarrollo está definido por las siguientes herramientas que aseguran la estabilidad y escalabilidad a futuro

- Lenguaje: Java.
- Interfaz: JavaFX

- Persistencia: MySQL

5. Elicitación

Para hacer la elicitación se aplicaron los siguientes métodos para descubrir la realidad del negocio, entender qué necesita el complejo y ver qué problemas tienen los huéspedes a la hora de alquilar.

- **Entrevistas con el dueño:** En varias charlas con el dueño pudimos Identificar las necesidades. Durante las charlas con el dueño nos remarcó que él no ve a las personas que van a las cabañas como 'clientes' comunes, sino que los considera 'invitados', porque ***Buena Caza*** promueve una onda muy familiar y de hospitalidad. Por eso, para respetar la identidad de su negocio, decidimos usar la palabra 'Invitado' en todo este documento, en el diseño de las pantallas y en los nombres de las tablas de la base de datos.
- **Análisis de plataforma Wix actual:** Investigando en la información que hoy día se tiene en la página web actual, se detectó que la estructura de datos no permite la interacción automática del invitado para hacer reservas, junto con la falta de mantenimiento y configuración de la información necesaria lo que justifica el paso a una base de datos MySQL. Página web actual: `buenacazapehuenco.wixsite.com/home`

6. Conocimiento del negocio

El sistema será un reflejo digital preciso de las reglas operativas de la organización. ***Buena Caza*** opera con una dinámica de hospitalidad sobre el resto de las cosas y que exige una coordinación perfecta entre la logística de unidades y la estrategia de fidelización.

6.1 Reglas de Negocio

- Tarifas Temporales: Aplicación de precios según fecha de entrada y salida según temporada.
- Lógica de Fidelización (Comunidad ***Buena Caza***): Aplicación de descuentos del 10% a la gente que ya se hospedó en el complejo.

6.2 Modelo de Dominio conceptual

A continuación, el diagrama de modelo de dominio conceptual:

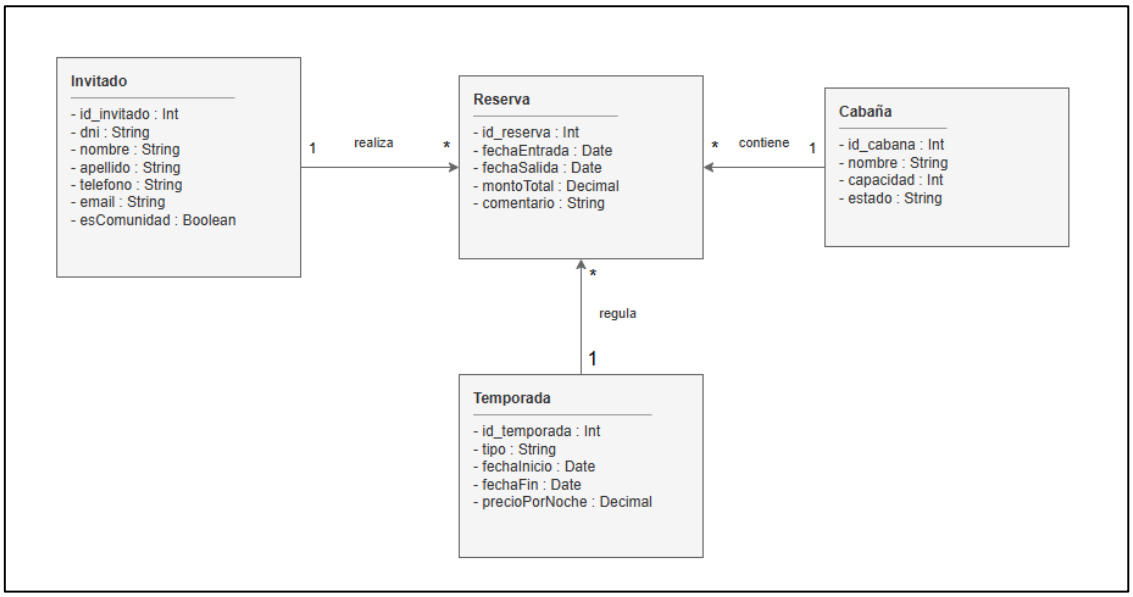


Figura 1: Modelo de dominio

- **Invitado:** Es la persona que viene a hospedarse. Guardamos sus datos de contacto y un campo clave llamado esComunidad (booleano) para saber si ya vino alguna vez y aplicarle el 10% de descuento automático de fidelidad.
- **Reserva:** Es el centro de todo el sistema. Registra las fechas de entrada y salida, cuánto tiene que pagar en total y una caja de comentarios por si viajan de a 2, 3 o 4 personas y necesitan ropa blanca extra.
- **Cabaña:** Representa físicamente a las 2 cabañas. Guardamos su capacidad máxima y si está disponible o bloqueada por mantenimiento.
- **Temporada:** Nos sirve para guardar las fechas y los precios por noche de las temporadas alta, media y baja, y así calcular los precios automáticamente.

7. Propuesta de solución

7.1 Arquitectura MVC

Se implementará una arquitectura **MVC (Modelo-Vista-Controlador).** Esta estructura es fundamental para la mantenibilidad, permitiendo que cambios en las reglas de precios (Modelo) no afecten directamente a la interfaz de usuario (Vista).

- **Capa de Vista (JavaFX):** Maneja las 8 pantallas con las que interactúan el Invitado y el Administrador, usando componentes interactivos como calendarios y listas.
- **Capa de Control (Java):** Contiene la lógica del negocio (validar contraseñas, calcular tarifas automáticas por temporada y chequear si corresponde el 10% de descuento por fidelidad).

- **Capa de Modelo (MySQL):** Es la base de datos que guarda los registros de los invitados, las temporadas configuradas y las reservas hechas. Se comunica con el controlador en Java mediante JDBC.

7.2 Diagrama de Arquitectura MVC

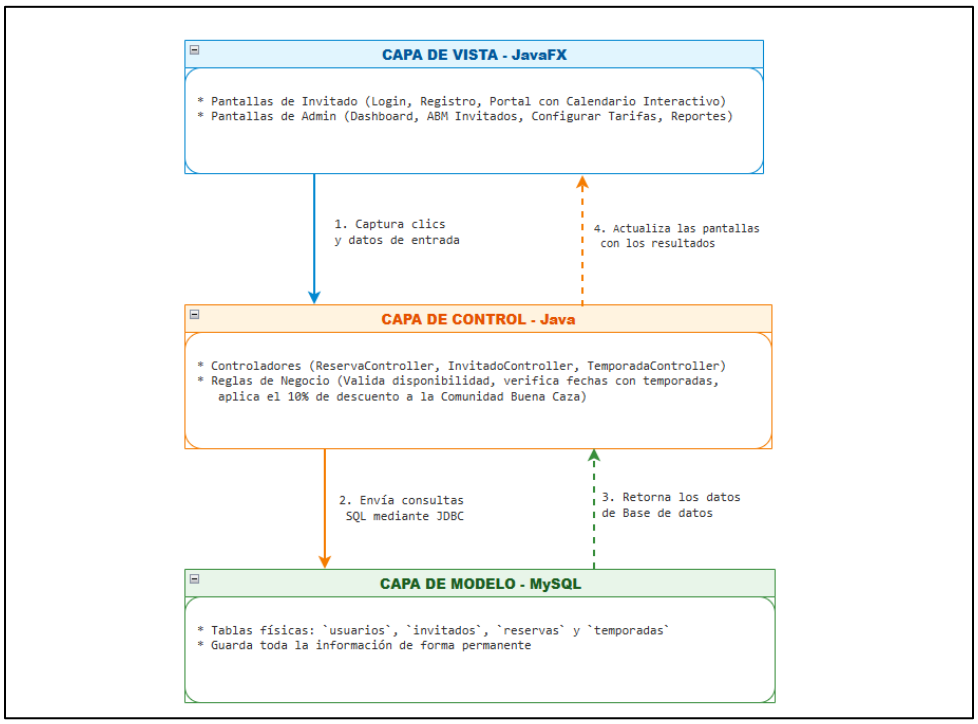


Figura 2: Diagrama de Arquitectura MVC

7.3 Descripción de la Interfaz Gráfica, entre paréntesis el rol

- **Login (admin/invitado):** Acceso mediante credenciales con validación de roles.
- **Registro de invitados (invitado):** Formulario para capturar datos personales y grabado la base de datos.
- **Pantalla de invitado (admin/invitado):** Pantalla de reservas. Uso de calendarios para selección de fechas y visualización de costos.
- **Pantalla de Administrador (admin)**: Resumen de la situación del complejo.
- **ABM de Invitados (admin)**: Gestión centralizada de la base de datos de invitado.
- **Configuración de Temporadas y precios (admin)**: Interfaz para ajustar los precios y fechas de temporadas.
- **Reporte de Ocupación (admin):** Listado de ocupación con filtros por unidad y temporada.
- **Reporte de Usuarios (admin):** Listado de los invitados de la ‘Comunidad Buena Caza’.

8. Inicio del análisis: casos de uso

Para modelar los requerimientos funcionales de la aplicación, armamos este diagrama UML de Casos de Uso. Nos sirve para ver de un vistazo qué personas (actores) interactúan con el software de ***Buena Caza*** y qué tareas específicas pueden realizar dentro del sistema:

- **Invitado (Actor Externo):** Puede registrarse solo (CU02), iniciar sesión de forma segura (CU01) y armar sus reservas en el calendario interactivo (CU03)
- **Administrador (Actor Interno):** El dueño puede loguearse (CU01), gestionar la base de datos de los huéspedes (CU05), definir tarifas/bloquear fechas por limpieza (CU04) e inspeccionar los reportes de ocupación e invitados (CU06 y CU07)

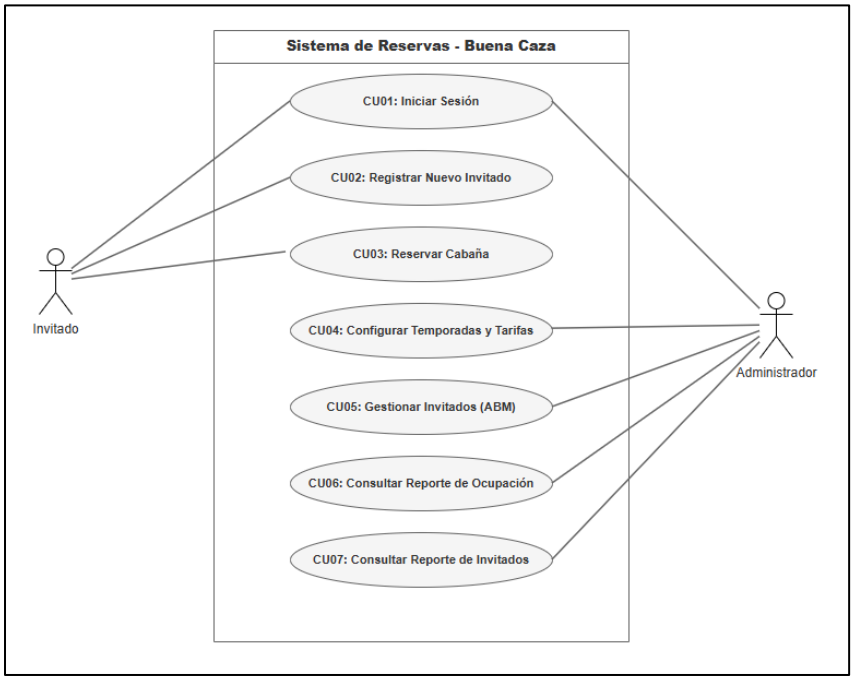


Figura 3: Diagrama de Casos de Uso General del Sistema

8.1 Identificación y Descripción de Actores

A continuación, se definen los actores:

- **Invitado**: Actor externo que consulta disponibilidad y genera transacciones de reserva.
- **Administrador**: Actor interno responsable de la administración y operativa del sistema.

8.2 Especificación de Requerimientos del Sistema

Dividimos las necesidades del sistema ***Buena Caza*** en requerimientos funcionales (qué hace el sistema) y no funcionales (atributos de calidad).

Requerimientos Funcionales

Requerimiento	Descripción
RFS01	El sistema debe requerir y validar un correo electrónico y una contraseña para permitir el acceso a perfiles de Invitado y Administrador.
RFS02	El sistema debe permitir el registro autónomo de nuevos invitados, capturando datos como DNI, Nombre, Apellido, Teléfono, Email y Contraseña.
RFS03	El sistema debe permitir al invitado poder realizar un alquiler elegir de forma interactiva entre las 2 cabañas.
RFS04	El sistema debe proveer selectores de fecha de entrada (Check-In) y salida (Check-Out) para delimitar la estadía según disponibilidad.
RFS05	El sistema debe calcular automáticamente el presupuesto de estadía según las tarifas configuradas para las temporadas (Alta, Media, Baja) que coincidan con las fechas elegidas.
RFS06	El sistema debe aplicar de forma automática un **10% de descuento de fidelidad** (Comunidad Buena Caza) si el usuario logueado ya cuenta con al menos una visita al complejo ***Buena Caza***.
RFS07	El sistema debe persistir (guardar) los datos de la reserva en MySQL asociándola unívocamente al ID del invitado de la sesión activa.
RFS08	El Administrador debe poder realizar el alta, modificación y baja (ABM) de invitados en la base de datos.
RFS09	El Administrador debe poder definir y guardar los rangos de fechas de las temporadas y sus correspondientes precios por noche para cada cabaña.
RFS10	El Administrador debe poder bloquear días específicos del calendario para inhabilitar reservas de una cabaña por razones de mantenimiento.
RFS11	El Administrador debe poder visualizar un reporte tipo lista con el resumen de noches ocupadas, facturación y porcentaje de ocupación por unidad y temporada.
RFS12	El Administrador debe poder consultar de forma centralizada un listado con todos los invitados registrados en el sistema, mostrando sus roles y datos de contacto.

Requerimientos No Funcionales

Requerimiento	Descripción
RNF01	El sistema de escritorio debe ser desarrollado completamente utilizando el lenguaje de programación **Java**.
RNF02	La persistencia de la información del negocio debe ser gestionada mediante un servidor de bases de datos relacionales **MySQL**.
RNF03	La interfaz gráfica de usuario (GUI) del sistema debe construirse utilizando la biblioteca de desarrollo **JavaFX**.
RNF04	El diseño de código del sistema debe estructurarse bajo la arquitectura **Modelo-Vista-Controlador (MVC)** para separar la lógica de negocio del diseño visual.
RNF05	La comunicación y ejecución de consultas entre la app Java y la base de datos MySQL se debe realizar por medio de la API estándar de **JDBC**.
RNF06	La interfaz de reservas para la Visita debe ser altamente intuitiva, permitiendo completar el cálculo del presupuesto y la reserva en un máximo de 5 clics.

Tabla de Trazabilidad

A continuación, se muestra una tabla donde mapeamos cada requerimiento funcional con su respectivo caso de uso general, asegurando que ninguna necesidad del negocio se quede sin cobertura en el desarrollo

Requerimiento	Caso de Uso Asociado	Actor	Descripción
RFS01	CU01: Iniciar Sesión	Invitado / Admin	Control de acceso seguro y carga de rol en el sistema.
RFS02	CU02: Registrar Nuevo Invitado	Invitado	Alta de usuario y cliente en MySQL.
RFS03, RFS04, RFS05, RFS06, RFS07	CU03: Reservar Cabaña	Invitado	Búsqueda, cálculo de tarifas, descuentos e inserción de la reserva.
RFS09, RFS10	CU04: Configurar Temporadas y Tarifas	Administrador	Definición de precios por noche y bloqueos por mantenimiento.
RFS08	CU05: Gestionar Clientes (ABM)	Administrador	Control y edición manual de la base de datos de huéspedes.
RFS11	CU06: Consultar Reporte de Ocupación	Administrador	Reporte en lista plana del flujo de estadías en el complejo.
RFS12	CU07: Consultar Reporte de Invitados	Administrador	Listado completo de la Comunidad Buena Caza registrada.

Especificación Detallada de los Casos de Uso Claves

Se tomaron 3 de los casos de uso más relevantes (CU01, CU3 y CU04) para su detalle.

Caso de uso CU01

	Caso de uso CU01: Iniciar Sesión
Caso de Uso	**CU01: Iniciar Sesión**
Actores	Invitado / Administrador
Referencias	RFS01, RNF03, RNF04, RNF05.
Descripción	Permite a cualquier usuario registrado (sea Invitado o Administrador) ingresar al sistema validando su email y contraseña contra la base de datos MySQL, habilitando las pantallas correspondientes según su rol.
Precondición	El usuario debe estar registrado previamente en el sistema (sus datos deben existir en la tabla usuarios de la base de datos MySQL).
Flujo Principal (Paso a Paso)	**1.** El usuario abre la aplicación y el sistema muestra la pantalla de Login (VentanaLogin) **2.** El usuario ingresa su correo electrónico y su contraseña en los campos de texto correspondientes **3.** El usuario hace clic en el botón "Ingresar". **4.** El sistema (a través del controlador AutenticacionController) toma los datos ingresados de la pantalla.

	5. El controlador realiza una consulta a la tabla usuarios de MySQL buscando el email ingresado. **6.** El sistema recupera el registro del usuario con su contraseña y su rol asignado ("Invitado" o "Administrador"). **7.** El sistema valida que la contraseña ingresada coincida con la que está guardada en la base de datos. **8.** Al comprobar que los datos son correctos, el sistema inicializa la sesión activa guardando el rol y los datos del usuario **9.** El sistema cierra la pantalla de Login y redirige al usuario a su menú principal según sea su perfil **10.** Fin del caso de uso.
Postcondición	El sistema habilita el acceso a las funciones personalizadas de la aplicación según el rol detectado y mantiene la sesión activa en el sistema.
Flujos Alternativos	**A1 (Credenciales incorrectas - Paso 5 o 7):** Si el correo no existe en MySQL o la contraseña ingresada no coincide, el sistema frena el ingreso, muestra un mensaje de alerta indicando: *"Usuario o contraseña incorrectos"* y limpia el campo de contraseña **A2 (Campos vacíos - Paso 3):** Si el usuario hace clic en "Ingresar" sin haber completado alguno de los dos campos, el sistema detecta la falta de datos y le muestra un aviso en pantalla: *"Complete todos los campos antes de continuar"*, sin realizar ninguna consulta a la base de datos. **A3 (Error de conexión con la Base de Datos - Paso 5):** Si hay un problema de red o el servidor MySQL está offline, el controlador captura la excepción) y evita que el programa se cierre de golpe y muestra una alerta: *"No hay conexión con el servidor. Intente más tarde"*.
Excepciones	El usuario debe tener conexión a la base de datos MySQL; de lo contrario, no podrá loguearse.

Caso de uso CU03

	Caso de uso CU03: Reservar Cabaña
Actores	Invitado
Referencias	RFS03, RFS04, RFS05, RFS06, RFS07, RNF03, RNF04.
Descripción	Permite que un cliente con sesión activa elija una cabaña y fechas, visualice el presupuesto neto con los descuentos de fidelidad aplicados de forma automática, y asocie una nueva reserva a su cuenta.
Precondición	El usuario debe estar registrado en el sistema y haber iniciado sesión de forma exitosa (CU01).
Flujo Principal	**1.** El invitado ingresa a la opción "Portal de Reservas" del menú. **2.** El sistema detecta la sesión activa y muestra en pantalla un saludo personalizado con su Nombre y Apellido. **3.** El invitado selecciona la unidad que desea reservar (Waingunga o Seeonee) desde un selector desplegable **4.** El invitado elige los días de su estadía mediante dos calendarios para fecha de entrada (Check-In) y fecha de salida (Check-Out). El sistema deberá calcular automáticamente el presupuesto al terminar de definir la fecha de salida.

	5. El sistema realiza una consulta en la base de datos MySQL para recuperar la tarifa por noche que corresponda según la temporada configurada para ese rango de fechas **6.** El sistema verifica que ningúno de los dias del rango de fecha este bloqueado **7.** El sistema consulta la tabla reservas de MySQL buscando alquileres previos del ID del invitado logueado. Al comprobar que es un cliente recurrente, calcula de forma automática el **10% de descuento** sobre el subtotal **8.** El sistema muestra el detalle como: cantidad de días, subtotal de días, descuento aplicado (10% Comunidad Buena Caza) y Costo Total. **9.** El sistema inserta el nuevo registro en MySQL y muestra un mensaje en pantalla indicando: *"¡Reserva generada con éxito!"*. **10.** Fin del caso de uso.
Postcondición	La cabaña cambia su estado a "Reservada" para las fechas seleccionadas en MySQL, impidiendo que otros usuarios la elijan, y la reserva se archiva vinculada al cliente.
Flujos Alternativos	**A1 (Falta de Disponibilidad - Paso 6):** Si el sistema detecta que la cabaña seleccionada tiene fechas que se superponen con otra reserva activa o fechas (por ejemplo, por superposición de fechas con otra estadía ya confirmada o días bloqueados por mantenimiento) o con días bloqueados por mantenimiento (RFS10), detiene el proceso, muestra una alerta flotante indicando: *"Cabaña no disponible para las fechas seleccionadas"* y regresa al paso 3 para que el cliente modifique su consulta. **A2 (Invitado Nuevo sin Historial - Paso 7):** Si la consulta MySQL en la tabla reservas devuelve cero registros de estadías previas para ese cliente, el sistema establece el descuento de fidelidad en $0.00, informa amigablemente en pantalla que el descuento Comunidad se aplicará en su próxima visita y continúa con el flujo normal en el paso 8 **A3 (Caída de Conexión Base de Datos - Durante consultas):** Si ocurre un error de conectividad de red con el motor MySQL, la app Java captura la excepción JDBC (SQLException), evita la caída inesperada del software y muestra un aviso emergente al usuario recomendándole intentar de nuevo en unos minutos.
Excepciones	No contempla pasarela de cobro integrada; el pago de la seña se pacta externamente.

Caso de uso CU04

	Caso de uso CU04: Configurar Temporadas y Tarifas
Actores	Administrador
Referencias	RFS09, RFS10, RNF03, RNF04.
Descripción	Permite al administrador definir los rangos de fechas para las temporadas (Alta, Media, Baja), definir las tarifas correspondientes por noche para cada cabaña (Waingunga y Seeonee) y bloquear días específicos del calendario por tareas de mantenimiento o limpieza.
Precondición	El Administrador debe estar registrado en el sistema y haber iniciado sesión correctamente (CU01).
Flujo Principal	**1.** El administrador selecciona la opción "Tarifas y Temporadas" desde el menú del Panel de control de JavaFX (Pantalla 4)

	2. El sistema carga desde la base de datos MySQL los rangos de temporadas vigentes y los muestra en la grilla **3.** El administrador elige el tipo de temporada que desea configurar ("Alta", "Media" o "Baja") desde un selector desplegable **4.** El administrador selecciona la "Fecha de Inicio" y la "Fecha de Fin" de la temporada mediante los calendarios **5.** El administrador introduce las tarifas por noche correspondientes en los campos de texto para cada uno de las cabañas **6.** El administrador presiona el botón "Guardar Temporada" **7.** El sistema valida que los campos de precios contengan valores numéricos válidos y positivos (mayores a cero) **8.** El sistema realiza una consulta en MySQL para verificar que el nuevo rango de fechas no se superponga con una temporada del mismo tipo ya configurada **9.** Al pasar las validaciones, el sistema inserta o actualiza de forma transaccional los datos en la tabla temporadas de MySQL **10.** El sistema actualiza automáticamente la grilla visual de la pantalla y emite un mensaje flotante confirmando: *"Configuración guardada correctamente"* **11.** Fin del caso de uso.
Postcondición	Las nuevas tarifas y límites temporales quedan registrados en MySQL, afectando de manera directa y automática a los cálculos de presupuestos que realicen los clientes en el portal de reservas (CU03).
Flujos Alternativos	**A1 (Rango de fechas superpuesto - Paso 8):** Si el sistema detecta que las fechas seleccionadas coinciden o se cruzan con otra temporada ya cargada en MySQL, frena el guardado, resalta los campos de los calendarios en rojo y muestra una alerta: *"Error: El rango de fechas seleccionado se superpone con una temporada existente"*. El flujo vuelve al paso 4. **A2 (Datos de precios no válidos - Paso 7):** Si el administrador ingresa letras, caracteres especiales o números negativos en los campos de texto de las tarifas, el sistema detecta la inconsistencia antes de enviar la consulta, muestra un mensaje de advertencia *"Ingrese un monto numérico válido"* y limpia los campos de texto para que los vuelva a escribir.
Excepciones	Caída del motor de base de datos MySQL durante la actualización: el sistema captura la excepción, cancela la transacción para no corromper los datos, y avisa al administrador que revise la conexión del servidor.

9. Aplicación del proceso unificado de desarrollo (PUD)

9.1 Etapa de Análisis

En esta etapa profundizamos en cómo interactúan los componentes del sistema para resolver las tareas principales de Buena Caza. Pasamos de las necesidades del negocio expuestas en el TP1 a un esquema técnico que detalla la lógica de software.

9.1.1 Diagrama de Secuencia

Diseñamos el diagrama de secuencia para el caso de uso central CU03: Reservar Cabaña.
En este diagrama se muestra cómo viajan los mensajes en orden cronológico entre el invitado, la interfaz de JavaFX, el controlador Java y la base de datos MySQL.

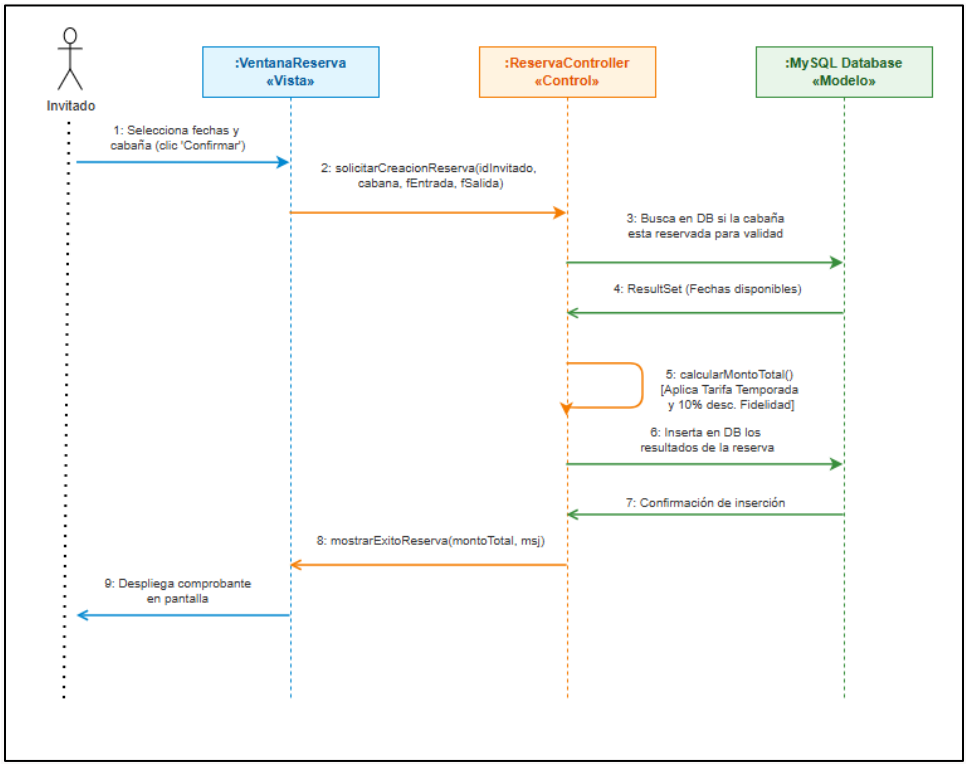


Figura 4: Diagrama de secuencia

9.1.2 Descripción del flujo de mensajes entre Vista, Control y Entidad

- **Entrada de datos en la Vista (JavaFX):** El invitado elige las fechas y la cabaña en la interfaz `VentanaReserva` y presiona "Confirmar". La pantalla captura los datos y llama al método `solicitarCreacionReserva()` del controlador.
- **Validación y Lógica en el Control (Java):** El `ReservaController` consulta a MySQL con un `SELECT` para verificar que la cabaña esté libre. Si hay disponibilidad, calcula el monto total cruzando la época con la tabla de temporadas y aplica el 10% de descuento si es cliente frecuente.

- **Persistencia y Respuesta en la Base de Datos (MySQL):** El controlador ejecuta un `INSERT` en la tabla `reservas` mediante JDBC. MySQL confirma la grabación, el controlador recibe la respuesta y la Vista le despliega el comprobante digital al invitado.

9.2 Etapa de Diseño

En esta fase transformamos el análisis conceptual en un diseño de software ejecutable. Definimos la estructura interna de las clases orientadas a objetos, sus atributos privados, visibilidades y métodos con parámetros.

9.2.1 Diagrama de Clases de Diseño

Modelamos las clases del sistema divididas por estereotipos UML: Vistas (`Boundary`), Controladores (`Control`) y Entidades de Datos (`Entity`).
Este esquema muestra los miembros (- privado, + público) y las multiplicidades de las relaciones que coinciden con nuestra base de datos.

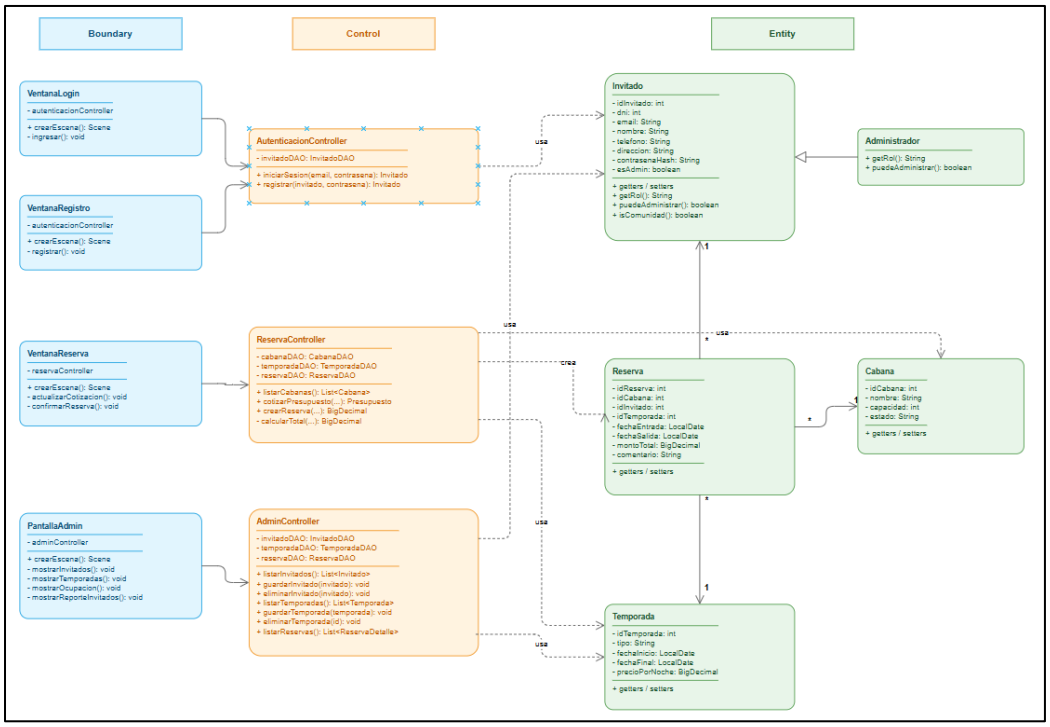


Figura 5: Diagrama de clase de diseño

9.2.2 Descripción de clases, métodos e interfaces del diseño

- **Clases de Vista («Boundary»):** VentanaLogin, VentanaRegistro, VentanaReserva y PantallaAdmin son las pantallas hechas en JavaFX. Muestran los datos, toman lo que hace el usuario y se lo pasan a su controlador. Cada pantalla tiene su controlador como atributo.

- **Clases de Control («Control»):** AutenticacionController se encarga del inicio de sesión y del registro de invitados. ReservaController arma el presupuesto (temporada, cantidad de noches y 10% de descuento para la Comunidad Buena Caza) y crea la reserva. AdminController maneja el ABM de invitados, las temporadas y los reportes. Para leer y guardar en MySQL, cada controlador usa los DAO que tiene como atributos.
- **Clases de Entidad («Entity»):** Invitado, Administrador, Reserva, Cabana y Temporada representan los datos del negocio. Tienen sus atributos privados con métodos get y set. Administrador hereda de Invitado y sobrescribe getRol() y puedeAdministrar(). Cada reserva está asociada a un invitado, una cabaña y una temporada.
- **Clases que no se muestran en el diagrama:** para que el diagrama sea más simple no se incluyó la capa DAO (la interfaz Dao<T> y las clases CabanaDAO, InvitadoDAO, ReservaDAO y TemporadaDAO, que se comunican con MySQL usando JDBC) ni las clases de apoyo Conexion, PasswordHasher, ComponentesUI, ColumnaTabla, Presupuesto, ReservaDetalle y TemporadaTipo. Igual están en el código, y algunas se explican en la sección 11.

9.3 Etapa de Implementación

En esta etapa mostramos la distribución física del sistema, es decir, en qué equipos (hardware) y entornos de ejecución (software) va a correr la aplicación de Buena Caza. Definimos cómo se conecta el programa de JavaFX con la base de datos MySQL a través de la red local o internet.

9.3.1 Diagrama de Despliegue

Diseñamos un Diagrama de Despliegue UML que muestra los nodos físicos del sistema (el equipo del cliente/administrador y el servidor de datos) y cómo se comunican mediante la API JDBC por el puerto 3306.

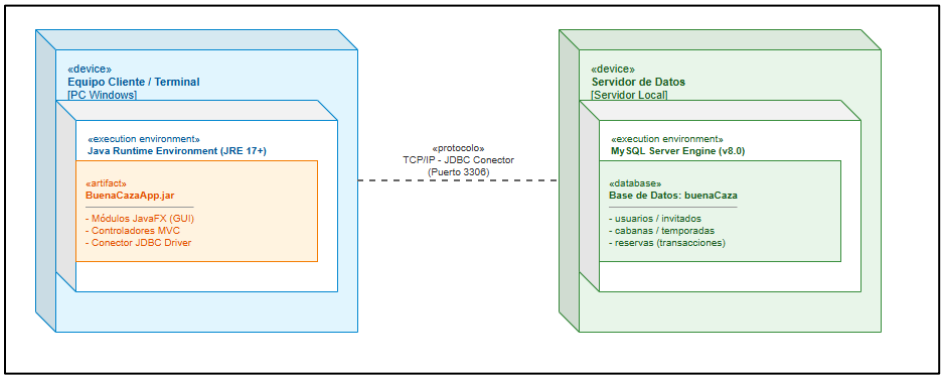


Figura 6: Diagrama de despliegue

9.3.2 Descripción de componentes y distribución física del hardware y software

- **Nodo Cliente / Terminal (`Equipo Cliente`):** Representa las computadoras de escritorio o laptops (Windows) utilizadas por el administrador o los invitados. Ejecuta el entorno **JRE 17+** donde corre el ejecutable **`BuenaCazaApp.jar`** con todas las pantallas en JavaFX.

- **Nodo Servidor de Datos (`Servidor MySQL`):** Es el equipo servidor (local en el complejo o alojado en la nube) que ejecuta el motor **MySQL Server v8.0**. Se encarga de procesar las consultas SQL y almacenar de forma persistente la base de datos `buenaCaza`.
- **Canal de Comunicación y Protocolo:** La conexión entre ambos nodos se realiza mediante red TCP/IP utilizando el driver **MySQL Connector/J (JDBC)** a través del puerto estándar **3306**.

9.4 Etapa de Pruebas

En este capítulo del trabajo planificamos las pruebas y verificamos los resultados de la implementación del prototipo de Buena Caza. Organizamos esta etapa en los cuatro artefactos del PUD: Plan de prueba, Modelo de pruebas, Tratamiento de defectos y Evaluación de prueba.

9.4.1 Plan de Pruebas

En este artefacto definimos las estrategias, recursos, tipos y niveles de prueba para garantizar la cobertura y validación funcional del sistema.

- **Niveles de Prueba:**
    - **Pruebas Unitarias:** Verificamos el correcto funcionamiento aislado de métodos individuales en Java, como el cálculo del 10% de descuento o si una fecha está disponible para alquiler.
    - **Pruebas de Integración:** Evaluamos la comunicación entre componentes, asegurando que los controladores Java ejecuten consultas SQL en MySQL sin fallas de conexión.
    - **Pruebas de Sistema:** Validamos el software completo de punta a punta, comprobando el flujo integral desde el Login hasta la generación del comprobante de reserva.
    - **Pruebas de Aceptación:** Realizamos simulaciones reales de uso junto al dueño de Buena Caza para confirmar que el sistema cumple con todas sus necesidades operativas.
- **Tipos de Prueba:**
    - **Pruebas Funcionales (Caja Negra):** Evaluamos las entradas y salidas del sistema en las pantallas sin revisar el código fuente, validando que se apliquen las reglas de negocio y restricciones.
    - **Pruebas No Funcionales:** Verificamos la usabilidad de la interfaz JavaFX, el tiempo de respuesta en las consultas de disponibilidad y la seguridad de las contraseñas encriptadas.
    - **Pruebas de Caja Blanca:** Analizamos la estructura interna del código Java para asegurar que se ejecuten todos los caminos lógicos y se manejen adecuadamente las excepciones y entre el correcto mensaje de error.

9.4.2 Modelos de Pruebas

El modelo de pruebas describe cómo se verifican los componentes ejecutables del sistema mediante dos elementos fundamentales: casos de prueba y procedimientos de prueba.

a) **Casos de Prueba:** Especifican la forma de probar el sistema mediante escenarios concretos, indicando datos de entrada, condiciones de ejecución y resultados esperados

ID	Caso de uso	Prueba	Datos de Entrada	Resultado Esperado
CP01	`CU01`	Login exitoso de usuario	Email: murbicain@gmail.com Pass: `Admin123*`	Inicia sesión correctamente y redirige a la pantalla de Administrador.
CP02	`CU01`	Credenciales inválidas	Email: `invitado@gmail.com` Pass: `ClaveErronea`	Bloquea el acceso y muestra alerta: *"Usuario o contraseña incorrectos"*.
CP03	`CU03`	Reserva exitosa con descuento	Cabaña: Waingunga Entrada: 0/01/2027 Salida: 15/01/2027	Graba la reserva en MySQL y aplica el 10% de descuento por cliente frecuente.
CP04	`CU03`	Intento de reserva en fecha ocupada	Cabaña: Waingunga Entrada: 12/01/2027 Salida: 18/01/2027	Detecta el solapamiento y muestra alerta: *"Cabaña no disponible en esas fechas"*.
CP05	`CU04`	Actualización de tarifa de temporada	Temporada: Alta Nuevo precio: `$85.000`	Modifica el valor en la tabla `temporadas` y actualiza los cálculos futuros.

b) **Procedimiento de Prueba:** Especificamos la secuencia detallada de pasos para ejecutar los casos de prueba sobre la interfaz gráfica de usuario JavaFX

- **Precondiciones:** Servidor MySQL activo con la base `buena_caza` cargada con datos iniciales y el ejecutable corriendo en Java.
- **Pasos de Ejecución:** El tester selecciona los controles visuales, ingresa las credenciales o fechas especificadas y presiona el botón de confirmación.
- **Verificación:** Se compara el mensaje emitido en pantalla y la persistencia en la base de datos contra el resultado esperado del caso.

9.4.3 Tratamiento de Defectos

Gestionamos el registro, seguimiento y resolución de cualquier defecto detectado durante la etapa de testing. Clasificamos los hallazgos en tres categorías conceptuales:

a) **Clasificación de Anomalías:**

- **Error:** Equivocación cometida por el desarrollador durante la escritura del código fuente.

- **Defecto:** Síntoma de problema o bug detectado internamente durante la ejecución del modelo de pruebas.
- **Falla:** Incumplimiento de un requerimiento observado por el usuario final durante el uso del sistema

b) **Registro y Control de Defectos:**
Documentamos cada defecto en una planilla indicando ID, módulo afectado, gravedad, desarrollador asignado y estado (Abierto / En corrección / Resuelto)

9.4.4 Evaluación de Prueba

Realizamos un análisis de los resultados obtenidos para determinar el nivel de calidad del software:

La siguiente tabla tiene como objetivo informar los resultados de las pruebas, permitiendo al responsable documentar cada ejecución y registrar cualquier problema o defecto detectado para su posterior resolución.

ID Prueba	Procedimiento	Observaciones	Fecha y Hora	Responsable	Resultado (Ok / Falla)
PRB-01-01	Ingresar usuario `murbicain@gmail.com` y contraseña en `VentanaLogin`.	Verificar acceso concedido y redirección inmediata al `PantallaAdmin`.	27/09/2026 09:00	M. Urbicain	**Ok**
PRB-01-02	Ingresar contraseña errónea en `VentanaLogin`.	Validar bloqueo de acceso y despliegue de alerta: *"Usuario o contraseña incorrectos"*.	27/09/2026 09:15	M. Urbicain	**Ok**
PRB-03-01	Seleccionar Cabaña Waingunga (10/01/2027 al 15/01/2027) con usuario Comunidad en `VentanaReserva`.	Validar cálculo automático de 5 noches en Temporada Alta con 10% de descuento ($382.500) y registro en la tabla `reserva` de MySQL.	27/09/2026 09:30	M. Urbicain	**Ok**
PRB-03-02	Intentar reservar Cabaña Waingunga del 12/01/2027 al 18/01/2027 (fechas solapadas).	Validar bloqueo por la lógica de control y despliegue de alerta: *"Cabaña no disponible en las fechas seleccionadas"*.	27/09/2026 09:45	M. Urbicain	**Ok**
PRB-04-01	Modificar tarifa de Temporada Alta a $85.000,00 en el panel de administración.	Confirmar actualización en la tabla `temporada` de MySQL y recálculo en cotizaciones del sistema.	27/09/2026 10:00	M. Urbicain	**Ok**

Resultados:

- **Cobertura de Casos de Prueba:** Se ejecutó el 100% de los casos planificados (5 de 5 casos probados exitosamente).

- **Cobertura de Requerimientos:** Se validaron los requerimientos funcionales RFS01, RFS03 y RFS09.
- **Estado de Defectos:** Se confirmó la resolución del 100% de los defectos de severidad alta antes de dar por finalizada la iteración.

9.5 Interfaz gráfica

Interfaz de acceso unificado donde los invitados y administradores ingresan sus credenciales para ingresar en el sistema.

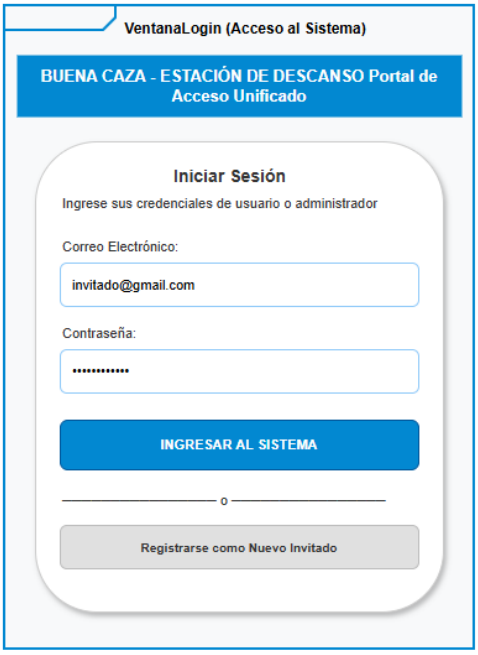


Figura 7: VentanaLogin

Pantalla para invitados que permite seleccionar la cabaña, las fechas de estadía y consultar el presupuesto y si le corresponde el 10% de descuento por fidelidad



Figura 8: VentanaReserva

Panel de control del administrador que consolida métricas clave, accesos a gestión y la tabla en tiempo real de las reservas registradas.

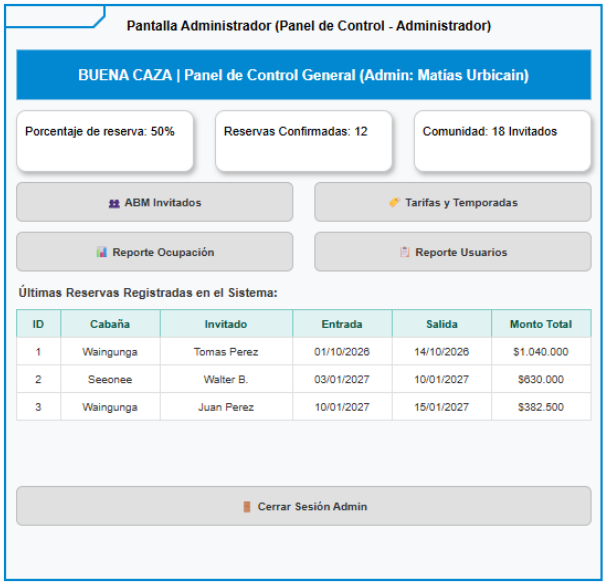


Figura 9: PantallaAdmin

10. Definición de la base de datos

10.1 Definición de la Base de Datos Relacional para el Prototipo

Para la persistencia de datos de Buena Caza implementamos una base de datos relacional en MySQL llamada `buena_caza`. La diseñamos bajo la Tercera Forma Normal (3FN) para evitar duplicidad de datos y garantizar la integridad referencial de reservas, huéspedes y tarifas.

10.2 Diagrama Entidad-Relación de la Base de Datos

El esquema relacional define la estructura física con claves primarias (`PK`) e introdujimos claves foráneas (`FK`) en la tabla central `reserva`. Esto conecta cada reserva con 1 invitado, 1 cabaña y 1 temporada específica. Tambien colocamos otra clave foránea (`PK`) en la tabla y en la tabla `temporada` para conectarla con la tabla `temporada_tipo`.

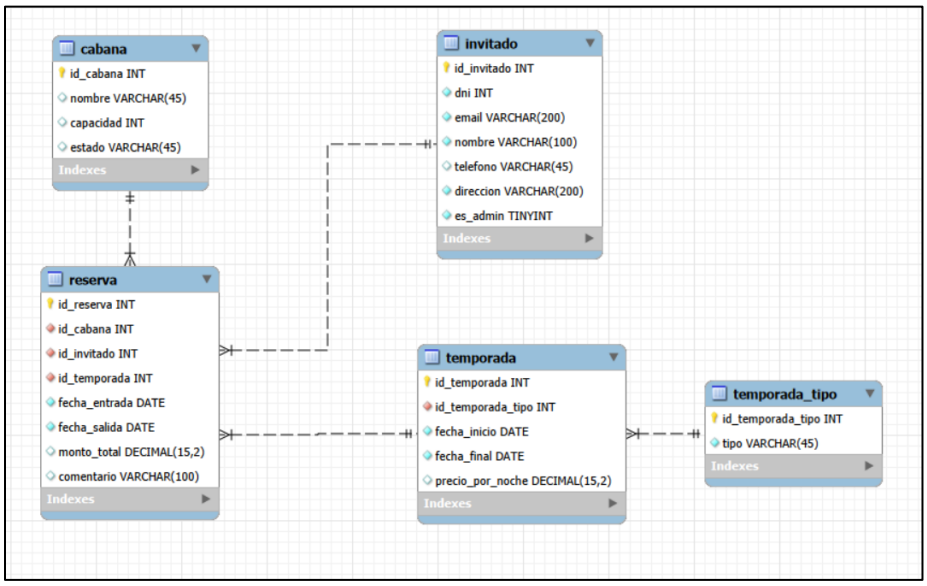


Figura 10: Diagrama entidad-relación

10.3 Creación de Tablas (CREATE TABLE)

Escribimos los scripts de SQL en formato `snake_case` para mantener el estándar y con motor `InnoDB` para la creación de la base de datos y sus tablas:

```
CREATE DATABASE  IF NOT EXISTS `buena_caza`
USE `buena_caza`;
```

```
--
-- Tabla`cabana`
--
CREATE TABLE `cabana` (
  `id_cabana` int NOT NULL AUTO_INCREMENT,
  `nombre` varchar(45) DEFAULT NULL,
  `capacidad` int DEFAULT NULL,
  `estado` varchar(45) DEFAULT NULL,
  PRIMARY KEY (`id_cabana`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
--
-- Tabla `invitado`
--
CREATE TABLE `invitado` (
  `id_invitado` int NOT NULL AUTO_INCREMENT,
  `dni` int NOT NULL,
  `email` varchar(200) NOT NULL,
  `nombre` varchar(100) NOT NULL,
  `telefono` varchar(45) DEFAULT NULL,
  `direccion` varchar(200) NOT NULL,
  `es_admin` tinyint NOT NULL DEFAULT '0',
  `contrasena_hash` varchar(255) DEFAULT NULL,
  PRIMARY KEY (`id_invitado`),
  UNIQUE KEY `uq_invitado_email` (`email`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
--
-- Tabla `reserva`
--
CREATE TABLE `reserva` (
  `id_reserva` int NOT NULL AUTO_INCREMENT,
  `id_cabana` int NOT NULL,
  `id_invitado` int NOT NULL,
  `id_temporada` int NOT NULL,
  `fecha_entrada` date NOT NULL,
  `fecha_salida` date NOT NULL,
  `monto_total` decimal(15,2) DEFAULT NULL,
  `comentario` varchar(100) DEFAULT NULL,
  PRIMARY KEY (`id_reserva`),
  KEY `id_cabana_idx` (`id_cabana`),
  KEY `id_invitado_idx` (`id_invitado`),
  KEY `id_temporada_idx` (`id_temporada`),
  CONSTRAINT `fk_reserva_cabana` FOREIGN KEY (`id_cabana`) REFERENCES `cabana`
(`id_cabana`),
  CONSTRAINT `fk_reserva_invitado` FOREIGN KEY (`id_invitado`) REFERENCES `invitado`
(`id_invitado`),
  CONSTRAINT `fk_reserva_temporada` FOREIGN KEY (`id_temporada`) REFERENCES `temporada`
(`id_temporada`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
--
-- Tabla `temporada`
--
CREATE TABLE `temporada` (
  `id_temporada` int NOT NULL AUTO_INCREMENT,
  `id_temporada_tipo` int NOT NULL,
  `fecha_inicio` date NOT NULL,
  `fecha_final` date NOT NULL,
  `precio_por_noche` decimal(15,2) DEFAULT NULL,
  PRIMARY KEY (`id_temporada`),
  KEY `id_temporada_tipo_idx` (`id_temporada_tipo`),
  CONSTRAINT `fk_temporada_temporada_tipo` FOREIGN KEY (`id_temporada_tipo`) REFERENCES
`temporada_tipo` (`id_temporada_tipo`)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
--
-- Tabla `temporada_tipo`
--
CREATE TABLE `temporada_tipo` (
  `id_temporada_tipo` int NOT NULL AUTO_INCREMENT,
  `tipo` varchar(45) NOT NULL,
  PRIMARY KEY (`id_temporada_tipo`)

) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci;
```

10.4 Inserción de Datos de Prueba (INSERT INTO)

Cargamos datos iniciales de configuración y para probar el prototipo: las dos cabañas del complejo (Waingunga y Seeonee), los tipos de temporada, las tarifas dinámicas, unos invitados de prueba junto con algunas reservas.

Tabla *cabana*

```
INSERT INTO `cabana` VALUES (1,'Waingunga',4,'libre'),(2,'Seeonee',4,'libre');`
```

Tabla invitado

```
INSERT INTO `invitado` (`id_invitado`, `dni`, `email`, `nombre`, `telefono`, `direccion`,
`es_admin`) VALUES (1,45454545,'matias@gmail.com','Matias Urbicain',NULL,'Alsina
291',1),(2,45253547,'tomas@gmail.com','Tomas Perez',NULL,'Sarmiento
501',0),(3,39224563,'walter@gmail.com','Walter Barrionuevo',NULL,'Salta 891',0);
```

Tabla *temporada_tipo*

```
INSERT INTO `temporada_tipo` VALUES (1,'Alta'),(2,'Media'),(3,'Baja');
```

Tabla *temporada*

```
INSERT INTO `temporada` VALUES (1,2,'2026-09-01','2026-12-01',80000.00),(2,1,'2026-12-
01','2027-03-01',90000.00),(3,3,'2027-03-01','2027-06-01',60000.00),(4,2,'2027-06-01','2027-
09-01',85000.00);
```

Tabla *reserva*

```
INSERT INTO `reserva` VALUES (1,1,2,3,'2026-10-01','2026-10-14',NULL,NULL),(2,2,3,1,'2027-
01-03','2027-01-10',NULL,NULL),(3,1,2,2,'2027-04-10','2027-04-24',NULL,NULL);
```

10.5 Consultas SQL de Gestión y Negocio (SELECT)

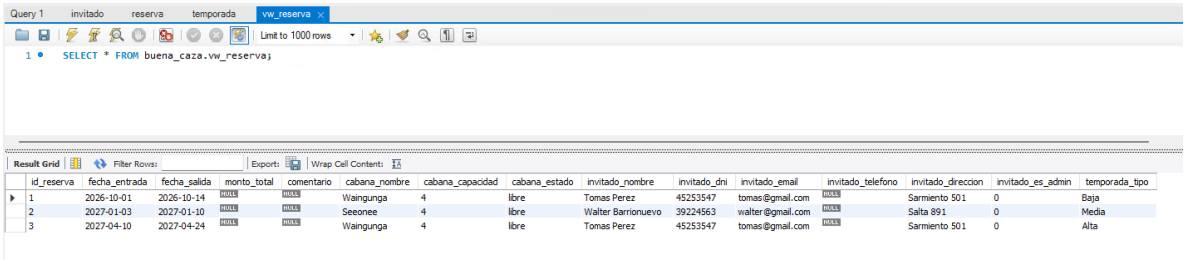
Definimos consultas esenciales para la lógica del sistema y creamos la vista `vw_reserva` para que con una sola consulta nos devuelve toda la información asociada a una reserva y que nos ayudará para el reporte de reservas.

```
CREATE VIEW `vw_reserva` AS
    SELECT
        `r`.`id_reserva` AS `id_reserva`,
        `r`.`fecha_entrada` AS `fecha_entrada`,
        `r`.`fecha_salida` AS `fecha_salida`,
        `r`.`monto_total` AS `monto_total`,
        `r`.`comentario` AS `comentario`,
        `c`.`nombre` AS `cabana_nombre`,
        `c`.`capacidad` AS `cabana_capacidad`,
        `c`.`estado` AS `cabana_estado`,
        `i`.`nombre` AS `invitado_nombre`,
        `i`.`dni` AS `invitado_dni`,
        `i`.`email` AS `invitado_email`,
        `i`.`telefono` AS `invitado_telefono`,
        `i`.`direccion` AS `invitado_direccion`,
```

```
    `i`.`es_admin` AS `invitado_es_admin`,
    `tt`.`tipo` AS `temporada_tipo`
FROM
    ((((`reserva` `r`
    JOIN `cabana` `c` ON ((`r`.`id_cabana` = `c`.`id_cabana`)))
    JOIN `invitado` `i` ON ((`r`.`id_invitado` = `i`.`id_invitado`)))
    JOIN `temporada` `t` ON ((`r`.`id_temporada` = `t`.`id_temporada`)))

    JOIN `temporada_tipo` `tt` ON ((`t`.`id_temporada_tipo` = `tt`.`id_temporada_tipo`)))
```

A continuación, una captura de la salida de la vista `vw_reserva`



Un script para consultar la disponibilidad de una cabaña

```
-- Consulta de verificación de disponibilidad de cabaña
SELECT * FROM `reserva` WHERE `id_cabana` = 1
    AND (`fecha_entrada` < '2027-01-20' AND `fecha_salida` > '2027-01-15');
```

10.6 Borrado y Limpieza de Registros (DELETE)

Definimos la query para borrar una reserva de prueba

```
DELETE FROM `reserva` WHERE `id_reserva` = 3;
```

10.7 Repositorio GitHub

Todo el código del proyecto está subido a GitHub: el código Java del prototipo (clases, pantallas y conexión con la base) y los scripts SQL para crear la base, las tablas y la vista, y para cargar los datos de prueba.

- **Repositorio de GitHub:** https://github.com/Murbbi/BuenaCaza.git

11. Desarrollo del prototipo en Java

En esta etapa se desarrolló un el sistema Buena Caza en Java, con interfaz gráfica en JavaFX y base de datos MySQL y conexión JDBC. El prototipo implementa los casos de uso principales definidos en el análisis: inicio de sesión (CU01), registro de invitados (CU02), reserva de cabañas (CU03), configuración de temporadas y tarifas (CU04), gestión de invitados (CU05) y los reportes de ocupación e invitados (CU06 y CU07).

A continuación, se explica cómo se abordó cada uno de los requisitos de la consigna, con fragmentos breves del código.

11.1 Estructura del proyecto

El código respeta la arquitectura MVC definida en la sección 7 y se organiza en los siguientes paquetes:

- **modelo:** clases de entidad (`Invitado`, `Administrador`, `Cabana`, `Reserva`, `Temporada`, `TemporadaTipo` y `ReservaDetalle`).
- **dao:** acceso a los datos mediante JDBC (interfaz `Dao` y clases `CabanaDAO`, `InvitadoDAO`, `ReservaDAO` y `TemporadaDAO`).
- **controlador:** lógica de negocio (`AutenticacionController`, `ReservaController` y `AdminController`).
- **vista:** pantallas en JavaFX (`VentanaLogin`, `VentanaRegistro`, `VentanaReserva` y `PantallaAdmin`).
- **conexion y seguridad:** conexión con MySQL (`Conexion`) y cifrado de contraseñas (`PasswordHasher`).
- **archivo:** exportación de datos a archivos de texto (`ExportadorTexto` y `ExportadorInvitados`).

11.2 Pilares de la programación orientada a objetos

11.2.1 Encapsulamiento

Los atributos de las clases son privados y solo se acceden mediante métodos públicos de lectura (get) y escritura (set). De esta forma ninguna otra clase puede modificar directamente el estado de un objeto:

```
public class Invitado {
    private int idInvitado;
    private int dni;
    private String email;
    private String nombre;
    private String telefono;
    private String direccion;
    private String contrasenaHash;
    private boolean esAdmin;
```

11.2.2 Herencia

`Administrador` hereda de `Invitado`: reutiliza todos sus atributos y métodos y solo agrega lo que lo diferencia. Con `super()` llama al constructor de la clase padre indicando que se trata de un administrador.

```java
public class Administrador extends Invitado {

    public Administrador(int idInvitado, int dni, String email, String nombre,
            String telefono, String direccion) {
        super(idInvitado, dni, email, nombre, telefono, direccion, esAdmin: true);
    }
```

En la base de datos ambos se guardan en la tabla `invitado` y `InvitadoDAO` crea un objeto u otro según la columna `es_admin`:

```java
        if (esAdmin) {
            return new Administrador(idInvitado, dni, email, nombre, telefono, direccion);
        }
        return new Invitado(idInvitado, dni, email, nombre, telefono, direccion);
```

11.2.3 Polimorfismo

`Administrador` sobrescribe (`@Override`) los métodos `getRol()` y `puedeAdministrar()` de `Invitado`. Al iniciar sesión, la aplicación recibe un objeto de tipo `Invitado` sin saber si es un administrador, y Java ejecuta la versión del método que corresponde a la clase del objeto para decidir qué pantalla mostrar:

```java
public class Invitado {


    public boolean puedeAdministrar() {
        return false;
    }
```

```java
public class Administrador extends Invitado {

    @Override
    public boolean puedeAdministrar() {
        return true;
    }
}
```

```java
  private void abrirSesion(Invitado invitado) {
      if (invitado.puedeAdministrar()) {
          mostrarAdmin(invitado);
          return;
      }
```

También se aplica polimorfismo por sobrecarga en los constructores de `Invitado` y `Temporada`, que comparten el nombre pero reciben distintos parámetros.

11.2.4 Abstracción

La interfaz `Dao<T>` define las operaciones que ofrece cualquier clase de acceso a datos (listar e insertar) sin definir cómo se resuelven. Cada DAO la implementa con su propio SQL, de esta forma las demás capas usan esas operaciones sin conocer los detalles de cada tabla:

```java
public interface Dao<T> {
    List<T> listar() throws SQLException;

    boolean insertar(T entidad) throws SQLException;
}
```

```java
public class CabanaDAO implements Dao<Cabana> {
    @Override
    public List<Cabana> listar() throws SQLException {
        String sql = "SELECT id_cabana, nombre, capacidad, estado FROM cabana ORDER BY id_cabana";
        List<Cabana> cabanas = new ArrayList<>();

        ....
    }

    @Override
    public boolean insertar(Cabana cabana) throws SQLException {
        String sql = "INSERT INTO cabana (nombre, capacidad, estado) VALUES (?, ?, ?)";

        ....
    }
}
```

11.3 Manejo de excepciones

Cuando algo no cumple una regla, el controlador lanza una `IllegalArgumentException` con un mensaje claro. Por ejemplo, si la fecha de salida no es posterior a la de entrada:

```java
    private void validarFechas(LocalDate entrada, LocalDate salida) {
        if (entrada == null || salida == null || !salida.isAfter(entrada)) {
            throw new IllegalArgumentException(s: "La salida debe ser posterior a la fecha de entrada.");
        }
        if (entrada.isBefore(LocalDate.now())) {
            throw new IllegalArgumentException(s: "La fecha de entrada no puede estar en el pasado.");
        }
    }
```

11.4 Menú de selección

El panel del administrador tiene un menú a la izquierda con todas las opciones del sistema cumpliendo con una de las consignas. Cada opción se agrega con su nombre y el método que muestra esa pantalla:

```java
        agregarOpcion(texto: "Gestionar invitados", this::mostrarInvitados);
        agregarOpcion(texto: "Temporadas y tarifas", this::mostrarTemporadas);
        agregarOpcion(texto: "Reporte de ocupación", this::mostrarOcupacion);
        agregarOpcion(texto: "Reporte de invitados", this::mostrarReporteInvitados);
        agregarOpcion(texto: "Panel de invitado", this::mostrarPanelInvitado);
```

12. Pantallas

En esta sección se muestran las pantallas del prototipo en funcionamiento, conectado a la base de datos MySQL `buena_caza` con los datos de prueba cargados. Las capturas recorren el inicio de sesión, el panel del administrador con su menú de opciones (gestión de invitados, temporadas y tarifas, y reportes) y el portal de reservas del invitado.

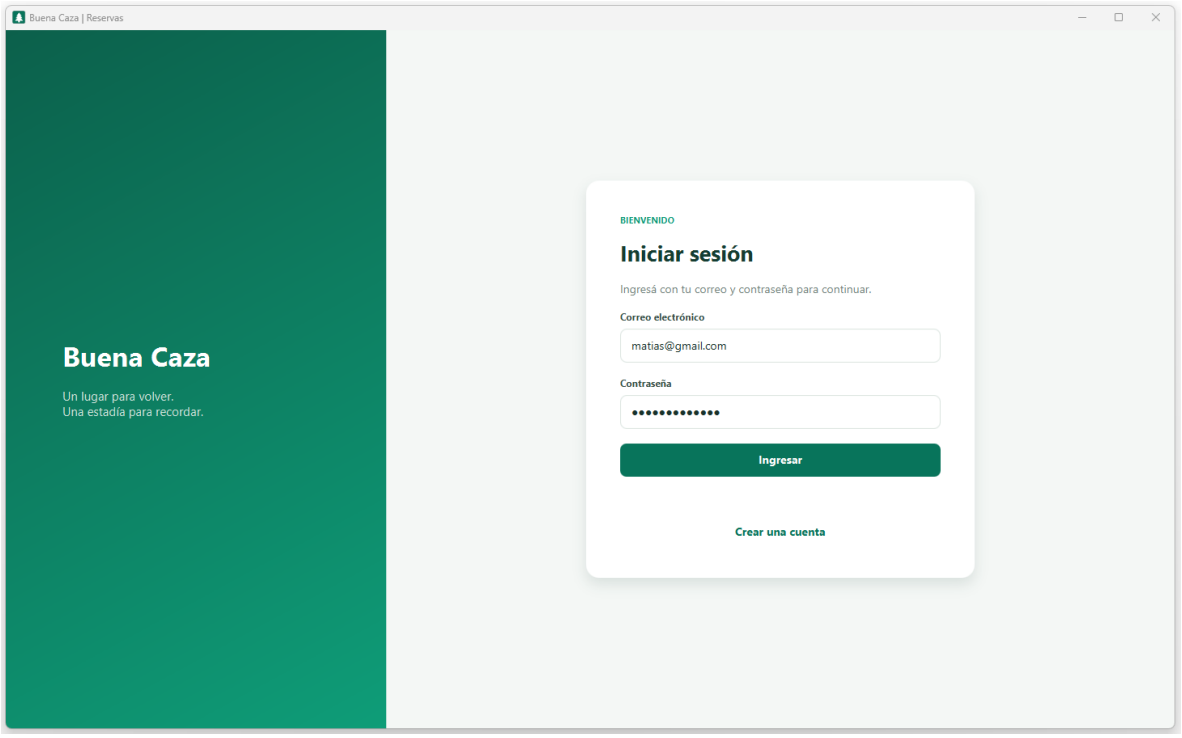


Figura 11: Inicio de sesión: acceso con correo y contraseña para todos los usuarios.

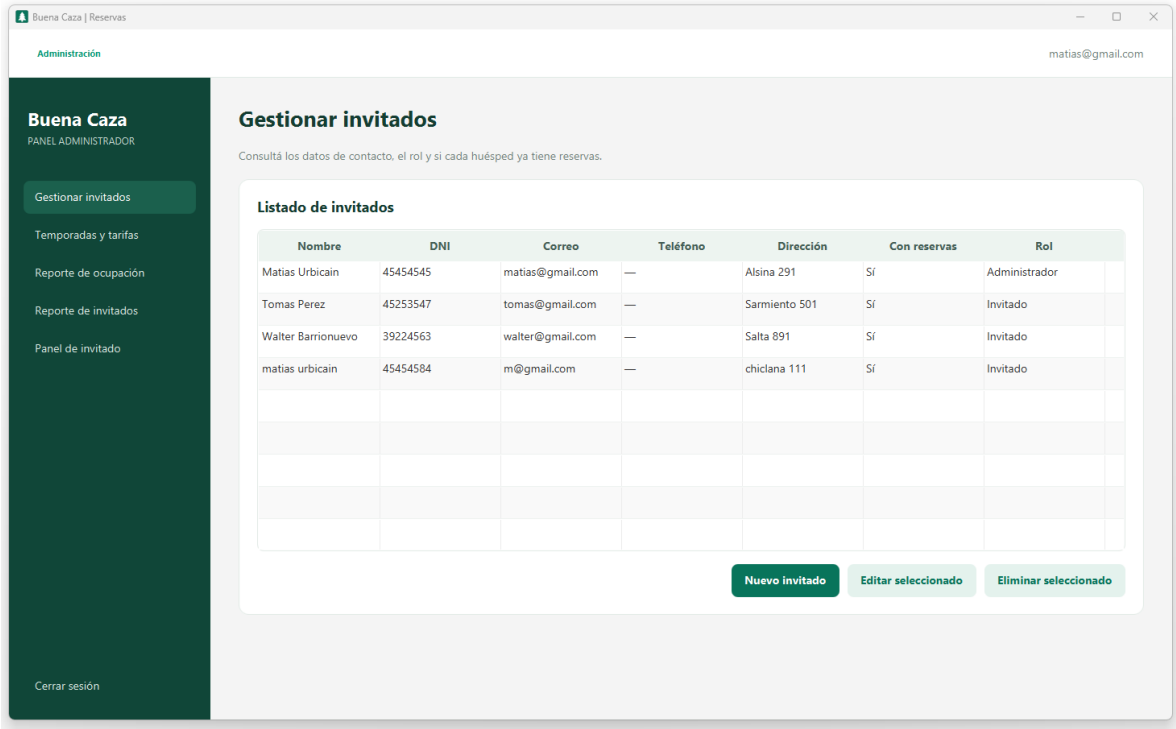


Figura 12: Gestión de invitados: listado con rol y opciones de alta, edición y baja.

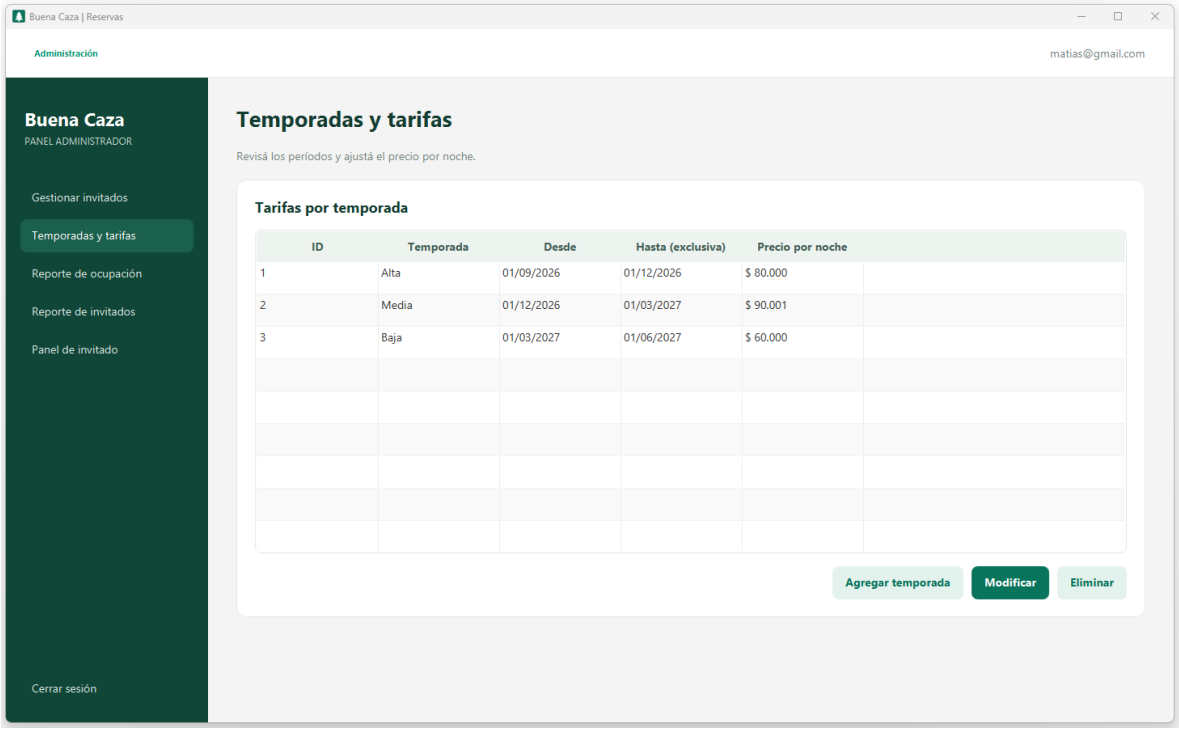


Figura 13: Temporadas y tarifas: períodos y precio por noche de cada temporada.

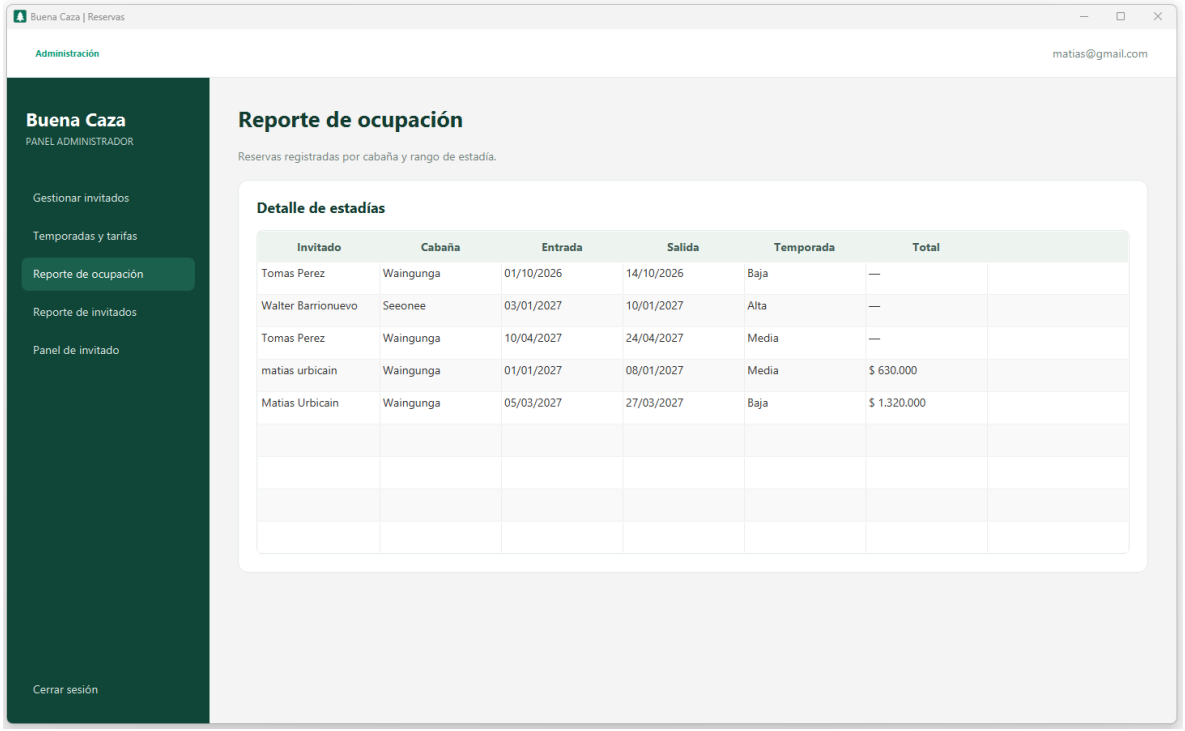


Figura 14: Reporte de ocupación: estadías registradas con cabaña, fechas y total.

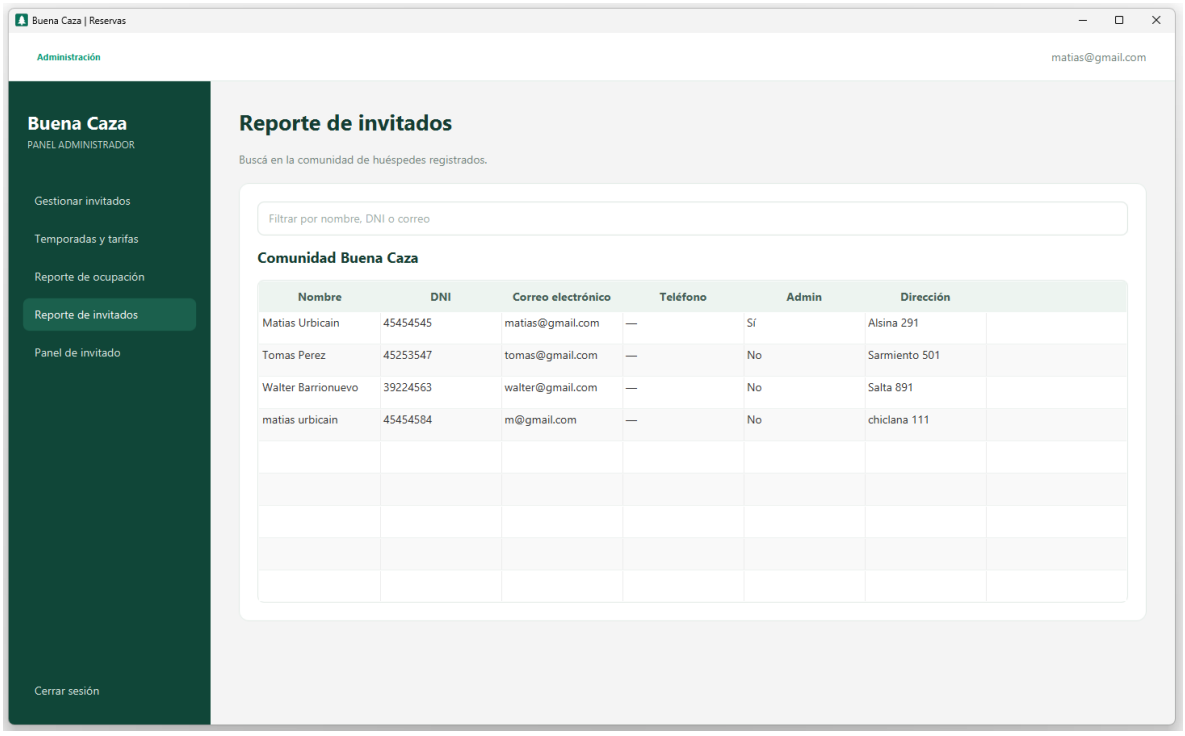


Figura 15: Reporte de invitados: Comunidad Buena Caza con filtro de búsqueda.

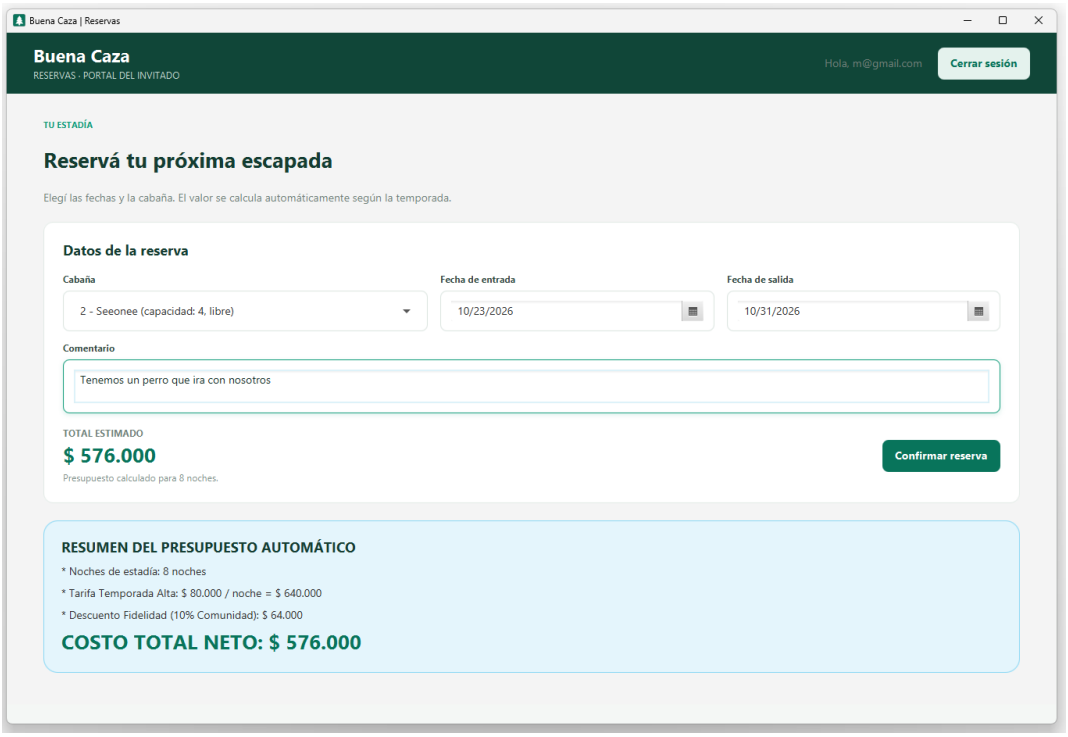


Figura 16: Portal del invitado: elección de cabaña y fechas con presupuesto automático.

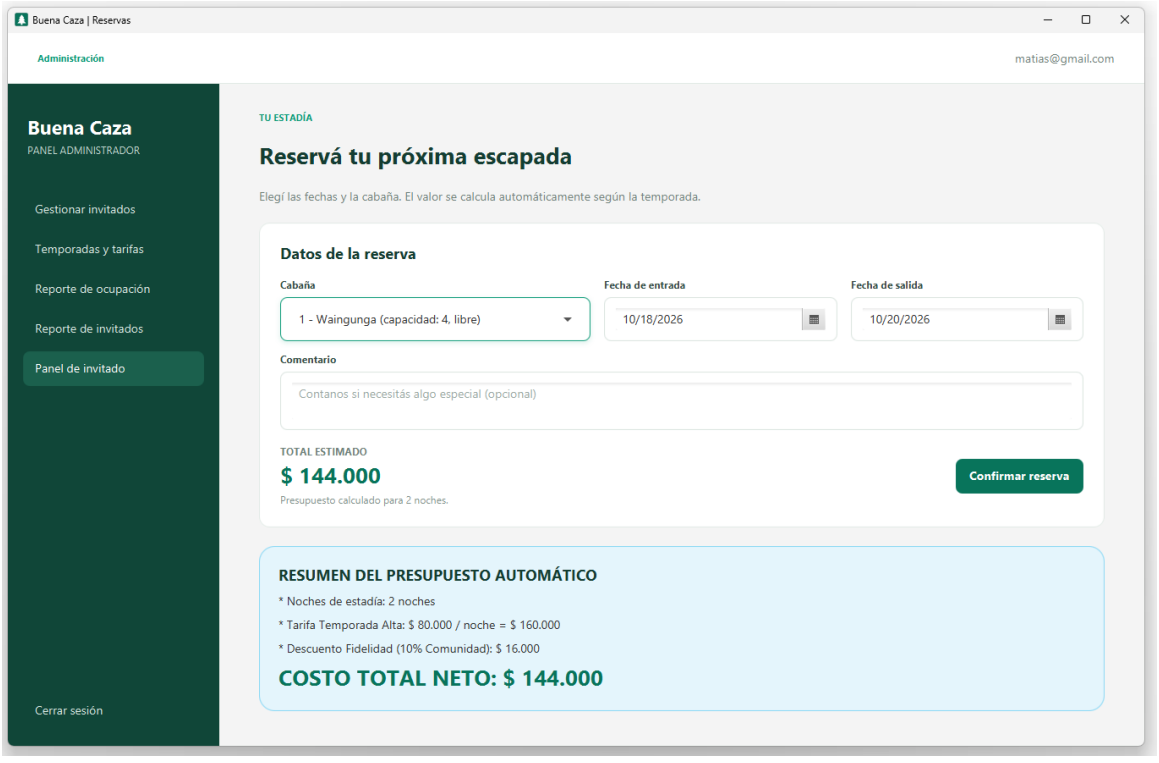


Figura 17: Panel de invitado: el administrador accede al formulario de reservas.

13. Versión final integradora (TP4)

En esta última entrega terminamos el prototipo de Buena Caza y lo cerramos como proyecto integrador. En este capítulo explicamos los puntos que pide el TP4: el patrón de diseño que elegimos, cómo guardamos y consultamos los datos en MySQL, cómo manejamos los errores con la base, dónde usamos interfaces y clases abstractas, y cómo usamos `ArrayList`. También agregamos algo nuevo al sistema: un botón para **exportar los clientes a un archivo de texto**, así cubrimos el uso de archivos.

Igual que en el TP3, no copiamos todo el código. Ponemos solo codigos cortos para que se entienda la idea, porque el código completo está en el repositorio.

- **Repositorio del proyecto (código y SQL):** https://github.com/Murbbi/BuenaCaza.git

- **Video de presentación:** https://youtu.be/fQVEFhU8Teg

13.1 Selección del patrón de diseño y justificación

Para `Buena Caza` elegimos el patrón **MVC (Modelo-Vista-Controlador)**. Ya lo habíamos dejado escrito como requerimiento no funcional en el análisis.

Requerimiento	Descripción
RNF04	El diseño de código del sistema debe estructurarse bajo la arquitectura Modelo-Vista-Controlador (MVC) para separar la lógica de negocio del diseño visual.

Así quedó repartido el código en las tres partes del patrón:

- **Modelo:** los paquetes `modelo` (Invitado, Administrador, Reserva, Cabana, Temporada) y `dao` (las consultas SQL). Es lo que maneja los datos con MySQL.
- **Controlador:** el paquete `controlador` (AutenticacionController, ReservaController y AdminController). Recibe lo que pide la vista, aplica las reglas del negocio (por ejemplo, el 10% de descuento a la Comunidad Buena Caza) y le pide los datos al modelo.
- **Vista:** el paquete `vista` con las pantallas en JavaFX (VentanaLogin, VentanaRegistro, VentanaReserva y PantallaAdmin). Solo muestra cosas y recibe los clics del usuario.

¿Por qué MVC? Porque si mañana el dueño del complejo quiere cambiar los colores o un botón, solo tocamos la vista. Y si cambiamos una tabla de la base, solo tocamos el DAO, sin miedo a romper las pantallas. Además, junto con MVC usamos el patrón **DAO** (Data Access Object): cada tabla tiene su clase que se encarga del SQL, así el controlador nunca escribe una consulta. Un ejemplo de cómo el controlador decide qué hacer y le pasa el trabajo al DAO:

```java
public void guardarInvitado(Invitado invitado) throws SQLException {
    if (invitado.getIdInvitado() == 0) {
        invitadoDAO.insertar(invitado);
    } else {
        invitadoDAO.actualizar(invitado);
    }
}
```

13.2 Persistencia y consulta de datos en MySQL

Para conectarnos a MySQL usamos solo **JDBC**, sin frameworks (como pide la materia). Dividimos el trabajo en los cuatro pasos que pide la consigna:

a) Establecer la conexión. La clase `Conexion` tiene un solo método que usa `DriverManager.getConnection()`. Los datos de acceso se pueden cambiar con variables de entorno, así no hace falta recompilar si cambia el servidor:

b) Realizar consultas. Cada DAO arma su `SELECT` con un `PreparedStatement`, recorre el `ResultSet` con un `while` y va creando objetos que guarda en un `ArrayList`:

```java
@Override
public List<Invitado> listar() throws SQLException {
    String sql = "SELECT id_invitado, dni, email, nombre, telefono, direccion, es_admin "
            + "FROM invitado ORDER BY id_invitado";
    List<Invitado> invitados = new ArrayList<>();

    try (Connection conexion = Conexion.getConexion();
            PreparedStatement sentencia = conexion.prepareStatement(sql);
            ResultSet resultado = sentencia.executeQuery()) {
        while (resultado.next()) {
            invitados.add(crearDesdeFila(resultado));
        }
    }

    return invitados;
}
```

c) Actualizar registros. Para dar de alta, modificar o borrar usamos `INSERT`, `UPDATE` y `DELETE` con `executeUpdate()`. Siempre con parámetros (`?`) y nunca pegando el texto del usuario en el SQL, así evitamos la inyección SQL:

```java
public boolean actualizar(Invitado invitado) throws SQLException {
    String sql = "UPDATE invitado SET dni = ?, email = ?, nombre = ?, telefono = ?, direccion = ? "
            + "WHERE id_invitado = ?";
    try (Connection conexion = Conexion.getConexion();
            PreparedStatement sentencia = conexion.prepareStatement(sql)) {
        sentencia.setInt(parameterIndex: 1, invitado.getDni());
        sentencia.setString(parameterIndex: 2, invitado.getEmail());
        sentencia.setString(parameterIndex: 3, invitado.getNombre());
        sentencia.setString(parameterIndex: 4, invitado.getTelefono());
        sentencia.setString(parameterIndex: 5, invitado.getDireccion());
        sentencia.setInt(parameterIndex: 6, invitado.getIdInvitado());
        return sentencia.executeUpdate() == 1;
    }
}
```

d) Presentar los resultados en la interfaz. La lista que devuelve el DAO pasa por el controlador y la vista la carga directo en una tabla `TableView`) Así, lo que se ve en pantalla es lo que hay en la base en ese momento (figuras 12 a 15 del capítulo 12):

```
List<Invitado> invitados = adminController.listarInvitados();

TableView<Invitado> tabla = new TableView<>(FXCollections.observableArrayList(invitados));
tabla.getColumns().add(ColumnaTabla.crear(titulo: "Nombre", Invitado::getNombre));
tabla.getColumns().add(ColumnaTabla.crear(titulo: "DNI", inv -> String.valueOf(inv.getDni())));
```

13.3 Manejo de excepciones con la base de datos

Cuando trabajamos con una base de datos pueden pasar muchas cosas: que el servidor esté apagado, que se corte la red o que el correo ya exista (la columna `email` es única). Si no lo manejamos, el programa se cierra de golpe. Por eso armamos el manejo en tres niveles:

- **En los DAO:** los métodos declaran `throws SQLException` y usan **try-with-resources**.
- **En los controladores:** dejan pasar la `SQLException` hacia arriba y además lanzan `IllegalArgumentException` cuando no se cumple una regla del negocio (por ejemplo, "no se puede eliminar un invitado que tiene reservas").
- **En la vista:** se atrapan con `try/catch` y se muestra un mensaje claro en una ventana, sin cerrar el programa. Cada tipo de error tiene su propio `catch` porque el mensaje para el usuario es distinto:

```
try {
    Invitado datos = new Invitado(nuevo ? 0 : seleccionado.getIdInvitado(),
            Integer.parseInt(dni.getText().trim()), email.getText().trim(), nombre.getText().trim(),
            telefono.getText().trim(), direccion.getText().trim());
    adminController.guardarInvitado(datos);
    mostrarInvitados();
} catch (NumberFormatException e) {
    ComponentesUI.mostrarError(stage, titulo: "Datos no válidos", mensaje: "El DNI debe tener solo números.");
} catch (SQLException e) {
    ComponentesUI.mostrarError(stage, titulo: "No se pudo guardar",
            "Revisá que el correo no sea de otra cuenta.\n" + e.getMessage());
}
```

13.4 Clases abstractas e interfaces

En el proyecto usamos las dos cosas, y cada una tiene un motivo:

Interfaz Dao<T>. Ya estaba desde el TP3. Dice qué operaciones tiene que tener cualquier clase de acceso a datos (`listar()` e `insertar()`) pero no dice cómo se hacen. `InvitadoDAO`, `CabanaDAO`, `ReservaDAO` y `TemporadaDAO` la implementan, cada una con su SQL. Usamos interfaz porque los DAO no comparten código, solo la "forma".

```
public interface Dao<T> {
    List<T> listar() throws SQLException;

    boolean insertar(T entidad) throws SQLException;
}
```

Clase abstracta ExportadorTexto<T> . Para el archivo de texto sí había código que se repite: abrir el archivo, escribir los títulos, acomodar cada columna a su ancho y recorrer la lista. Todo eso lo pusimos en una clase abstracta. Lo único que cambia según lo que se exporte es **qué datos va en**

cada columna, y eso queda como método abstracto. `ExportadorInvitados` hereda de ella y solo completa ese método:

```java
public abstract class ExportadorTexto<T> {

    protected abstract String[] valores(T elemento);

    public int exportar(List<T> datos, File destino) throws IOException {
```

```java
public class ExportadorInvitados extends ExportadorTexto<Invitado> {


    @Override
    protected String[] valores(Invitado invitado) {
        return new String[] {
                String.valueOf(invitado.getIdInvitado()),
                String.valueOf(invitado.getDni()),
                invitado.getNombre(),
                invitado.getEmail(),
                invitado.getTelefono(),
                invitado.getDireccion(),
                invitado.getRol(),
                conReservas.contains(invitado.getIdInvitado()) ? "Si" : "No" };
    }
```

La ventaja es que si el dueño después nos pide exportar también las reservas, alcanza con crear un `ExportadorReservas` con su método `valores()`, sin volver a escribir la parte del archivo.

13.5 Uso de ArrayList

Todo lo que viene de la base va en un `ArrayList`, porque no sabemos de antemano cuántos invitados, reservas o temporadas hay. El `ArrayList` crece solo a medida que agregamos elementos y nos da métodos simples como `add()`, `contains()` y `size()`. Lo usamos en todos los DAO (`listar()`, `listarDetalles()`, `listarTipos()`) y en la vista, por ejemplo para juntar los ids de los invitados que tienen reservas:

```java
    List<Integer> conReservas = new ArrayList<>();
    for (Invitado inv : invitados) {
        if (inv.isComunidad()) {
            conReservas.add(inv.getIdInvitado());
        }
    }
```

Ese mismo `ArrayList` de clientes es el que después se muestra en la tabla de la pantalla y el que se recorre para armar el archivo de texto de la exportación.

13.6 Uso de archivos: exportación de clientes a texto

Agregamos un botón **"Exportar a TXT"** en la pantalla **Reporte de invitados** del administrador. Sirve para sacar el listado completo de clientes a un archivo que podria usarse para tenerlo como respaldo. Elegimos el formato de **columnas de ancho fijo** porque se lee bien en cualquier editor y es fácil de importar.

```
try (BufferedWriter escritor = Files.newBufferedWriter(destino.toPath(), StandardCharsets.UTF_8)) {
    escritor.write(armarLinea(titulos));
    escritor.newLine();
    escritor.write("-".repeat(anchoTotal()));
    escritor.newLine();
```

Pantalla con el botón de exportación en Reporte de invitados



Figura 18: Botón de exportación en Reporte de invitados.

13.9 Resumen de los puntos que pedia la consigna de TP4:

Punto de la consigna	Dónde está en el código
Patrón de diseño	MVC + DAO. Paquetes `modelo`, `vista`, `controlador`, `dao`.
Conexión a MySQL	`conexion/Conexion.java` con JDBC y `DriverManager`.
Consultas	Métodos `listar()` de los DAO y la vista `vw_reserva`.
Actualizar registros	`insertar()`, `actualizar()` y `eliminar()` de los DAO.
Mostrar en la interfaz	`TableView` en `PantallaAdmin` de cabañas en `VentanaReserva`.
Excepciones con MySQL	`SQLException` + try-with-resources en los DAO y `try/catch` en la vista.
Interfaz	`dao/Dao.java` implementada por los 4 DAO.
Clase abstracta	`archivo/ExportadorTexto.java` extendida por `ExportadorInvitados`.
ArrayList	Resultados de todos los DAO y lista `conReservas`.
Archivos	Botón "Exportar a TXT": `BufferedWriter`, `File`, `IOException`.